

Replicação e Consistência

- Ideia: manter **múltiplas cópias** (réplicas) do mesmo dado/serviço para melhorar desempenho e disponibilidade, mas pagando o preço de **manter consistência** entre as cópias

Motivações para Replicação

- Desempenho
 - Cache (browser/proxy) reduz latência
 - Balanceamento de carga (DNS Round Robin)
 - Dados imutáveis são baratos de replicar; dados mutáveis aumentam a sobrecarga de atualização
- Disponibilidade
 - % do tempo que o serviço está acessível com resposta aceitável
 - Falhas comuns: servidor e particionamento de rede
 - Se n servidores falham independentemente com probabilidade p :
disponibilidade = $1 - p^n$
- Tolerância a falhas
 - Por colapso: em $f+1$ servidores, dá pra tolerar f falhas e ainda ter 1 ativo
 - Bizantinas: em $2f+1$ servidores, tolera f falhas arbitrárias via votação

Transparência e Arquitetura

- Transparência de replicação
 - Cliente não deve perceber múltiplas cópias (visão lógica única)
 - Operações devem retornar **um único resultado**, como se fosse 1 servidor
- Componentes
 - Gerenciadores de Réplica (GR): armazenam réplicas e executam operações
 - Front-end (FE): fica entre cliente e GRs e esconde a distribuição
 - Objeto lógico vs físico: 1 item lógico pode ter várias réplicas físicas
- Máquinas de estado (determinismo)

- Estado = função determinística do estado inicial + sequência de operações
- Para consistência forte, GRs não podem depender de coisas não determinísticas (ex: relógio local/sensores)

Processamento de Requisição (visão geral)

- FE envia requisição para 1 ou mais GRs
- GRs coordenam/definem ordem
- GRs executam
- GRs chegam em acordo sobre o efeito
- 1 ou mais GRs respondem ao FE

Comunicação em Grupo (Views)

- Membros podem entrar/sair (falha ou adição)
- Serviço de participação mantém a lista de membros ativos e notifica mudanças
- Views = listas numeradas que representam a membresia em um instante
- Comunicação virtualmente síncrona
 - Entre duas mudanças de view, todos entregam o mesmo conjunto de mensagens
 - Transferência de estado é essencial quando entra um novo GR

Problema do Comportamento Anômalo

- Atualizações podem “sumir” se um GR falha antes de propagar
- Resultado: cliente lê valores incompatíveis (violação da ideia de cópia única)

Modelos de Consistência

- Linearizabilidade
 - Critério mais rigoroso (“ouro”)
 - Ordem das operações deve respeitar **tempo real**
 - Se a escrita terminou antes da leitura começar, a leitura deve ver a escrita
- Consistência sequencial

- Respeita a **ordem do programa** de cada cliente
- Não respeita tempo real necessariamente (mais barata que linearizabilidade)
- Todo serviço linearizável é sequencial, mas não o contrário

Estratégias de Replicação

- Replicação passiva (Backup Primário)
 - FE fala só com o Primário
 - Primário executa, propaga estado/log para Backups, espera ACKs e então responde
 - Failover: se primário cai, um backup assume (e mantém consistência se a troca for “atômica” + IDs detectarem duplicatas)
 - Vantagens: simples e aceita não determinismo
 - Desvantagens: custo de rede (transferência de estado/log) e failover pode ser lento
- Replicação ativa
 - Todos GRs são pares
 - FE envia requisição via multicast confiável e totalmente ordenado
 - Todos executam na mesma ordem (precisa determinismo)
 - Failover “instantâneo” (outros já estão processando)
 - Para falhas bizantinas: coletar $f+1$ respostas iguais em $2f+1$ GRs (com assinaturas)

Disponibilidade Máxima (consistência relaxada)

- Ideia: responder mesmo com propagação “preguiçosa” (background)
- Garantia mínima: cada cliente se vê de forma consistente ao longo do tempo
- Gossip
 - GRs trocam atualizações periodicamente (lazy)
 - Suporta ordenação causal com carimbos vetoriais
 - Estruturas típicas: log de atualização, tabela de operações já executadas, carimbo/versão da réplica

- Bayou
 - Bom para conectividade variável (mobile)
 - Atualizações “tentativas” localmente (podem ser desfeitas/reordenadas)
 - Atualizações “confirmadas” com ordem canônica definida por um primário
 - Conflitos podem ser resolvidos por regras da aplicação
- Coda (sistema de arquivos)
 - Estratégia otimista para desconexão
 - Usa CVV (vetor de versões) por arquivo
 - Conflito quando CVVs não são comparáveis; pode exigir intervenção manual

Controle de Concorrência e Quórum

- Capacidade de serialização de uma cópia
 - Efeito das transações deve ser equivalente a execução serial em 1 conjunto de objetos (ACID)
 - Controle clássico: 2PL (travamento em duas fases)
- Um lê / todos escrevem (Read-One/Write-All)
 - Lê em 1 réplica
 - Escreve em todas (se 1 estiver fora, escrita bloqueia)
- Quórum (Gifford)
 - Réplicas têm votos/pesos
 - Leitura exige R votos; escrita exige W votos
 - Regras: $W > (\text{total}/2)$ e $R + W > \text{total}$ (garante interseção)